Table 7.4 Typical information in a bill of materials

Type of material used	*Tools and dies used*
Part name	Part identification number
Parent product	Quantity needed
Classification number	Type of standard part
Measuring units used	Process plan number
Part source	Part weight
Version number	Validity status
Production status	Value of part
Drawing number	Drawing format
Part description file	

7.14 Interfaces for CAD/CAM

Since the introduction of the computer as an automation tool for planning and control of manufacturing operations, the problem of interconnecting various software and hardware systems has become a major issue. The first computer-based planning and control systems were custom designed and configured for dedicated applications. It was extremely difficult and often impossible to take existing software and hardware modules developed for one plant and use them for the configuration of a planning and control system for another plant. This problem had been recognized many years ago and initiated standardization activities in almost every industrialized country. Also an attempt was made to draft standards on an international level to conceive standard interfaces for interconnecting CAD and CAM systems in order to transfer design data to the planning activities of manufacturing. Information which is communicated across the interfaces contains graphical, drawing related, geometry and product model data as in Figure 7.24, also see Schilli *et al.* (1989). This data is used for process planning, scheduling, manufacturing and quality control.

The concept of a universal data exchange system is depicted in Figure 7.25. In this figure it is shown how various CAD systems communicate with the production planning and control system. The adaptation c. protocols, data formats and data transmission rates is done by the interface. The interface must provide electrical and physical compatibility and it must ensure that the semantics of the exchanged information is maintained. Data conversion may be done directly in the interface with preprocessors or postprocessors. With the help of such an interface it becomes possible to use existing software and hardware modules for various applications in a factory. In addition, a company's own manufacturing data can be easily merged with those from vendors and suppliers. It is also possible to build manufacturing planning and control systems from heterogeneous modules supplied by different vendors.

There are certain requirements which a standardized interface must be capable

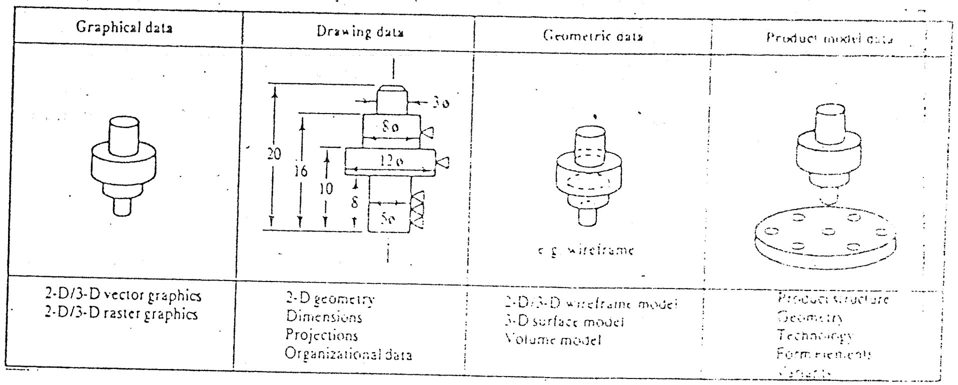

Figure 7.24
Product information communicated between CAD/CAM activities.

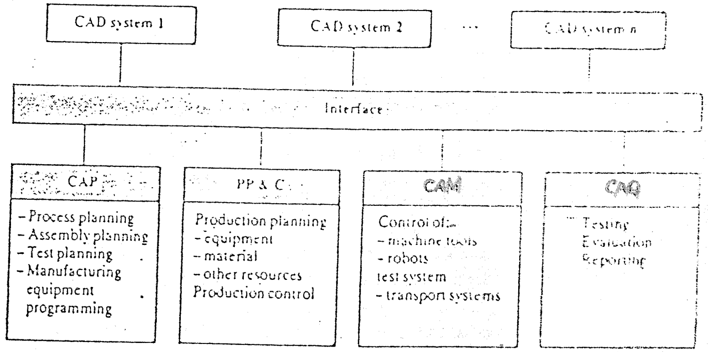

Figure 7.25
Universal interface for interconnecting the computer planning and control activities of a manufacturing system.

of providing. They are:

- The interface must be capable of handling all manufacturing data.
- There should be no information loss when data is transferred between heterogeneous systems. In other words, it must be possible to maintain the semantics during conversion.
- The system must be efficient to be capable of handling the realtime requirements of a manufacturing system.
- The system should be open-ended to permit extensions or contractions.
- The system should be adaptable to other standards.
- The system must be independent of the computer and communication architecture used.

- The number of data entities to be handled should be a minimum.
- It must be possible to form application-oriented subsets of the standard to reduce cost and overhead.
- A logic decoupling of data structure and physical memory format is necessary to ensure a separation of the specific information structure and the transmitted data units.
- The system should be capable of achieving production data.
- The interface must be upward- and downward-compatible in a hierarchical control structure.
- Test procedures must be provided to verify the efficiency and accuracy of data transmission.

7.15 Types of Interfaces

An interface can be regarded as an aggregate of conditions, rules and conventions which describes the information exchange, between two communicating objects. An object in our context can be human, software, modules, computer hardware or manufacturing processes. In principle, three types of interfaces can be distinguished including language interfaces, procedural interfaces and descriptive interfaces (Figure 7.26). The CAD system must be capable of exchanging data with the manufacturing world, and the interfaces are the windows to this world (Figure 7.27). In this figure, the interconnection of the CAD module with the various manufacturing system components are shown.

The language or user interface (Figures 7.26 and 7.27) is the window through which the user can communicate with the CAD module. The user employs the icons, pictorials, menus or textual languages to describe design and manufacturing problems. The textual languages are still the most important means of user/system communication. However, the interactive graphic languages are increasingly gaining importance, they are easier to learn and offer a more natural method of user/agent communication.

The procedural interfaces refer to language interfaces which are necessary to do the technical calculations, CAD modeling, process planning and machine programming. These interfaces may use subprograms, parameter descriptions or host languages to perform the individual CAD activities; thereby the host language concept is of particular importance. A host language is an existing language such as Fortran which has been extended to be able to handle specific constructs like the description of a circle. This is often done via subroutine calls. Procedural interfaces are also used to set up, in a generic way, the CAD configuration to be used or to manipulate data of the CAD database.

Figure 7.28 shows the concept of the Graphical Kernel System (GKS) which is a standard interface that can be used to operate different CAD hardware/software systems through an application program. GKS defines a language independent

	Language interface	Procedural interface (program interface)	Descriptive interface (data interface)
Types	Language constructs Language syntax Language grammer	Subprograms (of extended languages) Parameter description Host languages	Data structures Data formats
Representatives	Fortran, COBOL, Pascal • Data manipulation languages • Manufacturing machine programming language • Form element and variant description languages	GKS-graphic interface Data manipulation interface Fortran variant programming . . .	IGES SET VDAFS Libraries Archives . . .
Examples	P1 = POINT/0, 0, 12 L3 = LINE/P1, ATANGL, 45 C2 = CIRCLE /P1, P2, P3	CALL POINT (X, Y, Z, P1) CALL LINE (P1, 0) CALL CIRCLE (P1, P2, P3, C2)	'P1'421, 0, 0, 12 'L3'520, 0, 0, 12, 45 'L2'200, 0, 0, 12, 4, 5 3, 5, 6, 10

Figure 7.26
Types of software interfaces (Courtesy of Institute RPK University of Karlsruhe, Germany).

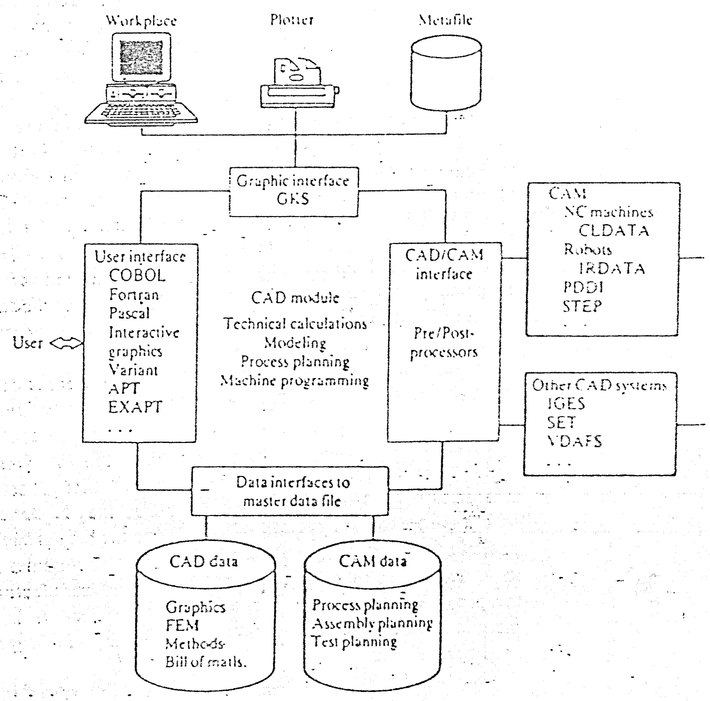

Figure 7.27
CAD/CAM interfaces.

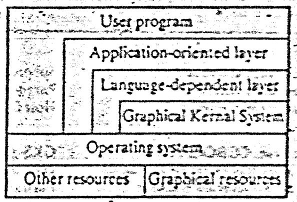

Figure 7.28 The layer model of the Graphical Kernel System (GKS).

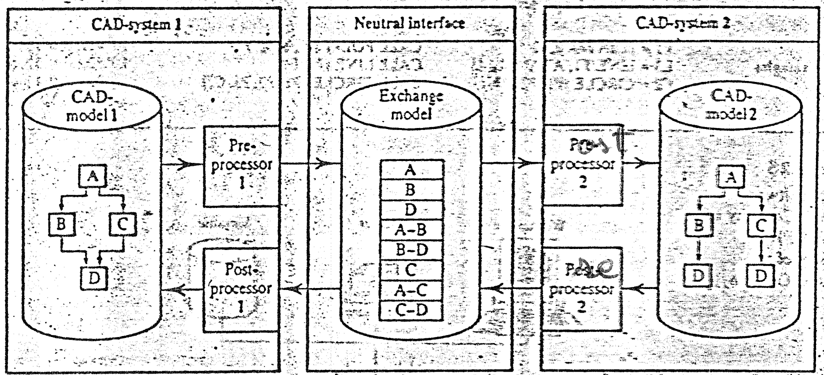

Figure 7.29 Exchange of CAD models between two different CAD systems.

nucleus for operating a graphic system. The user communicates with the graphic system via his program. The GKS system is embedded in the user program by an application-oriented layer and a language independent layer, whereby each layer calls upon the functions of the adjacent lower layer. The following services are available to the user:

- methods for the abstract definition of the peripherals which are needed for a CAD session;
- definition of basic graphic elements, polynomials, text and so on;
- performance of graphic operations such as transformations, rotations, picture segmentation, editing and so on;
- generation of a neutral picture file (meta-file) for the system independent storage of data, the exchange of data between different CAD systems and the organization of the output of graphic data;
- definition of a segment file for storing picture segments during a CAD session;
- input and output of graphic information.

Descriptive interfaces are the window to other CAD systems, to the master databases or to the manufacturing activities. The basic concept of a descriptive interface is shown in Figure 7.29, see also Grabowski and Glatz (1986). With this

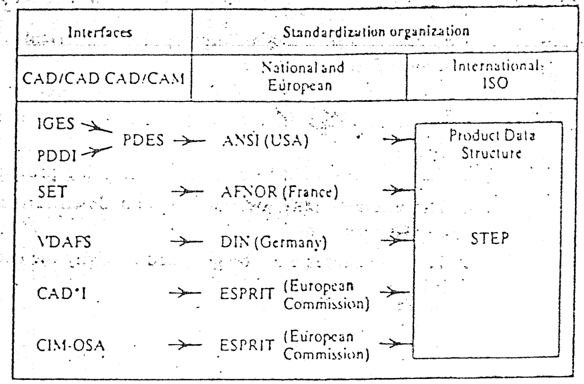

Figure 7.30 Development of CAD/CAD and CAD/CAM standards.

concept it is possible to exchange models between two different CAD systems. For example, model data of the CAD system A is converted via a pre-processor to a neutral presentation form and from there via post-processor to the representation scheme needed by the CAD system B. The exchange model representation is a system-independent method of describing the application. It can be accessed by any CAD system which has the proper pre- and post-processors. Typical equipment independent interfaces to manufacturing are CLDATA for NC machine programming and IRDATA for robot programming; they are explained elsewhere in the book.

In the following sections CAD and CAD/CAM interfaces will be discussed in more detail. The activities described will eventually lead to the product data structure defined by STEP (Figure 7.30).

7.16 Description of Various Interfaces

7.16.1 IGES (Initial Graphics Exchange Specification)

The basic concept of IGES is concerned with the exchange of product description data from one CAD system to another one as explained in IGES (1988). This exchange takes place via a neutral data format, see Figure 7.29. If information has to be transferred from one type of CAD system to another it is converted via a pre-processor to the neutral data format specified by IGES and then converted via a post-processor to the proper data format of the other CAD system. With this concept, it is possible to transfer CAD data between any set of CAD systems, as long as they are equipped with the corresponding pre- and post-processors.

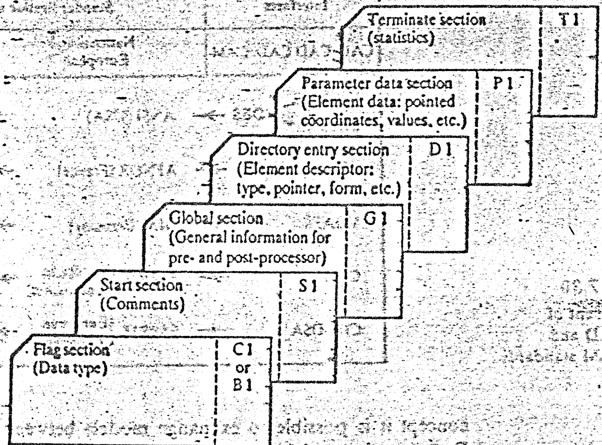

Figure 7.31
Sections of the IGES Datafile
(Source: NIST)

IGES tries to represent technical drawings; three-dimensional wireframe, surface and volume models; FEM models; and symbolic representations in a neutral data format.

The IGES concept developed to date has the following five versions:

Version	Year
IGES – Version 1.0	(1981)
IGES – Version 2.0	(1983)
IGES – Version 3.0	(1984)
IGES – Version 4.0	(1986)
IGES – Version 5.0	(1988)

Version 1.0 was conceived for the early CAD systems and was tailored toward the technology of the 1970s. Presently, most equipment manufacturers support Versions 2.0 and 3.0.

IGES contains records of 80 character length which are presented by the 80 columns card format; where columns 1–72 hold information in ASCII code and columns 73–80 hold an alphabetic character followed by a serial number to indicate sections.

An IGES file has six parts, including a flag section, start section, global section, directory entry section, parameter data section, and terminal section, see Figure 7.31.

- The flag section indicates if the file is in binary or in a compressed ASCII format. It may not be used when the contents are represented by ASCII characters. If the binary presentation is selected, the file can be compacted by 70%.

Object-related entities		Structure entities	Predefined associations
Geometric entities	**Modeling entities**		
Circular arc	Finite element	Associativity definition	Group w/o backpointers
Composite curve	Nodal	Associativity instance	External reference file index
Conic arc	Displacement/rotation	Drawing	Views visible
Copious data	Offset surface	Line font definition	Views visible, color
Plane	Curve on a parametric surface	Macro definition	Line weight
Line	Trimmed (parametric) surface	Macro instance	Entity label display
Parametric spline curve	Nodal results	Property	Single parent association
Parametric spline surface	Element results	Subfigure definition	External reference file index
Point		Network subfigure definition	Dimensional geometry association
Ruled surface	CSG-model	Singular subfigure instance	
Surface of revolution		Rectangular array subfigure	Ordered group w/o backp.
Tabulated cylinder	Block	Instance	Planar association
Transformation matrix	Right angle wedge	Circular array subfigure instance.	Flow
Flash	Right circle cylinder	Network subfigure instance	
Rational B-spline curve	Right circle cone	Text font definition	Drafting entities (annotations)
Rational B-spline surface	Frustum	View	
Offset curve	Sphere	External reference	Angular dimension
Connect point	Torus	Nodal load/constraint	Centerline
	Solid of revolution	Text display template	Diameter
	Solid of linear extrusion	Absolute text display template	Dimension
	Ellipsoid	Incremental text display template	Flag note
	Boolean tree	Color definition	General label
	Solid instance	Attribute table definition	General note
	Solid assembly	Attribute table instance	Leader (arrow)
			Linear dimension
			Ordinate
			Dimension
			Point dimension
			Radius dimension
			General symbol
			Sectionated area
			Section
			Witness line

Figure 7.32
Elements of IGES Version 4.0

- The start section contains comments; it provides human readable comments keyed in by the user and contains information for the receiving station.
- The global section holds information describing the pre-processor and data needed by the post-processor to handle the file.
- The directory entry section describes all IGES entities specified in the file (Figure 7.32). The entities are the various geometric elements including points, curves, surfaces and relations which are collections of similar structured entities; annotations like dimensions, viewing angles and notations, and structure elements like line font definitions, macro definitions, drawing specific data and so on.
- In the parameter data section, the parameters of the specified entities are stored in a free format.
- The terminate section contains a statistic indicating the size of each section which is represented by the last sequence number of a section.

Although IGES is a commonly used interface it has many problems. Version 1.0 was criticized for its huge file size and redundancy. This problem was reduced in Version 2.0 by introducing the binary format for setting up files. A problem exists of repeated definitions between the data entry section and the parameter data section, and also files are not very readable.

The pre- and post-processors are frequently of insufficient quality because companies who supply them try to maintain proprietary rights. Thus a one-to-one functional match between the transferred information of two CAD systems is often not obtained. Also the more recent versions of IGES have problems. For example, the IGES Version 3.0 was not developed to present information on solids. Thus solids must be described in an awkward method by boundary entities, and thus some information about solids cannot be transferred. In addition, there are representation schemes which are not unique and have unstructured file formats.

For the evaluation of the efficiency of interfaces a test matrix is used. Figure 7.33 shows in the upper part an input matrix for the CAD system and in the lower part the output to an NC programming system, see also Milberg and Peiker (1987). The 2-D geometry is transmitted in its entirety whereas the dimensions and text are missing.

7.16.2 PDDI (Product Definition Data Interface)

The PDDI interface was developed in connection with the ICAM project which was funded by the Air Force in order to improve the manufacturing methods in the aircraft industry, see also Weiss (1984). The essential goal of this product was the creation of an interface between CAD and CAM. PDDI is design-oriented, it provides a formal specification of a product model and its submodels. The latter represent:

(1) The 3-D geometry of the product which is defined by a set of curves, surfaces and volumes.

(2) The topology which describes the set of elements and relations for defining the product boundary.

(3) The tolerance which is inherent in the accuracy of the manufacturing processes.

(4) The future elements needed for defining manufacturing operations.

(5) The non-geometric information for organizational purposes and description of material and its properties.

Additional information includes the specification of the pre- and post-processors needed for the data exchange between CAD and CAM. The PDDI efforts were conceptual research activities and the results were used for the development of the PDES activities and are a major input to STEP. The STEP concept is described in Section 7.16.7.

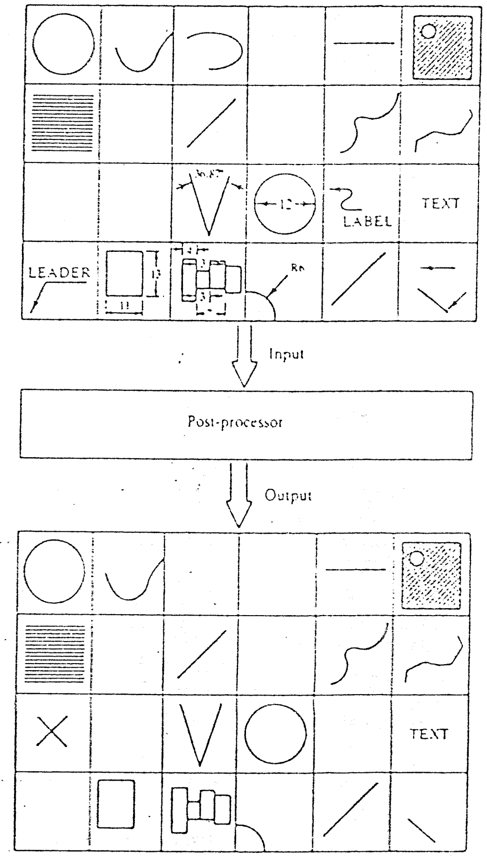

Figure 7.33
Evaluation of a post-processor with the help of a test matrix (Courtesy of Institut für Werkzeugmaschinen and Betriebswissenschaften, Technical University of Munich).

7.16.3 PDES (Product Data Exchange Specification)

The aim of PDES is to provide an interface which permits the exchange of data of the entire product development cycle and production cycle, see also PDES (1985). It can be viewed as an expansion of IGES whereby organizational and technological data have been added. Functionally PDES will contain IGES Version 4.0. The physical and logical structure of both interfaces will be different. For this reason there will be a conversion program to transfer the IGES into the PDES format.

A special feature of the PDES development is the use of the formal language EXPRESS for modeling the product information. There will be capabilities for defining manufacturing features, FEM and special applications for civil engineering, electronics, ship building and so on. The results of the PDES project will be the major input to STEP.

7.16.4 SET (Standard d'Echange et de Transfert)

The interface SET is a standard for the exchange of all data which is generated for CAD/CAM. It is a French development and was first published in 1984 (SET 85). Several additions have been published since (SET F 88, SET S 88 and SET V 88).

The information transferred via the SET interface are wireframe, surface, B-representation and FEM models, as well as technical drawings and scientific data. Organizational, material property and tolerance information cannot be defined by individual models but they must be contained in the drawing for transfer.

SET allows the exchange of data between different CAD/CAM systems and also between CAD/CAM systems and central data banks. It features a much more general data structure than IGES; data is stored in a very compressed form. Thus the neutral file size is much smaller and the processing time is much shorter than for IGES.

The structure of a typical SET file is shown in Figure 7.34. The delimiters of the file are begin and end blocks. The BEGIN section contains administrative data and other information of a general type.

The END section describes the total number of blocks stored in the SET file. The file is structured from assemblies which contain a series of blocks; the blocks may be nested and hold the CAD information which has to be transferred. There

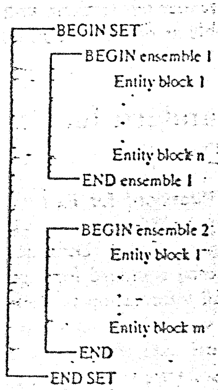

Figure 7.34 Structure of a SET data file.

are different types of information units defined. They are:

Class 0. The geometric primitives needed for describing 2- and 3-dimensional objects such as points, lines, circles, parabolas, planes and surfaces.

Class 1. Complex geometric entities for defining complex curves, complex solids, shells, faces and so on.

Class 2. Graphical aids to map product information onto technical presentations. These aids include view block definitions, block definitions, line fonts, cross hatching, characters for defining symbols and so on.

Class 3. Grouping mechanisms to assist the mathematical operations needed for the manipulation of the design objects. Typical representations are coordinate transformations, scalar series, general matrices, tabulated functions, physical quantities, material properties and so on.

Class 4. Descriptive aids to represent diameters, angles, labels, text, cross-hatching, surface finishes and so on.

Class 5. Elements defining the structural relations such as calling block, group, attribute and homogeneous entities.

Class 6. Connecting elements to describe logic between the elements of any class.

Class 7. FEM elements to build the FEM model. Typical elements are finite elements, model for an element, computations, definition of a computation, loads, damping, Eigenvalue, knot renumbering and so on.

Class 80. Gives the user the possibility of defining his own elements.

Class 99. Management aids to structure SET files. They include SET header, ensemble header, ensemble header as well as 'End' specifier for an ensemble and a 'SET'.

7.16.5 VDAFS (Verband der Automobilindustrie Flächenschnittstelle)

This interface was developed by the Association of the German Automobile Industry for the transfer of 3-D curves and surfaces for which IGES only offers an unsatisfactory solution. Two versions of this interface have been developed and are discussed in Frankfurt Verband der Automobilindustrie (1983) and Mund *et al.* (1987).

Similar to IGES, the VDAFS interface contains a fixed record length of 80 characters. A file consists of a header and a data section. The header describes the data source, the project name, the origination date, the validity date, the source CAD system and the user. In the data section the geometric objects are described with the help of entities. Characteristic for the data presentation is an APT-oriented data format.

Elements of the VDAFS Version 2.0 are shown in Figure 7.35. Curves and surfaces are described with the help of points, set of points and set of vectors (Figure 7.36).

Geometric elements	Non-geometric elements
Points	Begin specifier
Point sequences	Comment
Point vector sequences	Structuring (SET groups)
Circle (curve)	Transformation matrix
Polynomial curve	Transformation table
Surface	End specifier
Curve of surface	
Trimmed surface	
Surfaces of patches	

Figure 7.35
Elements of VDAFS Version 2.0.

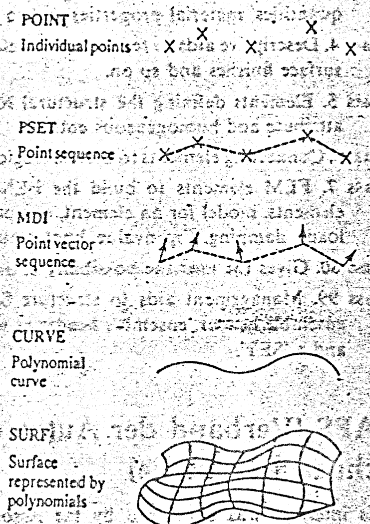

Figure 7.36
VDAFS geometry elements.

With the VDAFS interface it is possible to transfer any described workpiece surface, including the orientation of the cutting tool. Problems include the transfer of text, dimensions, cross-hatching and so on. In addition, single elements are received by the user in a basic atomic format and are difficult to use for further processing. Another problem is that the interface has insufficient means of structuring data.

7.16.6 CAD*I interface

The CAD*I interface was developed within the framework of the ESPRIT Project 322 of the European Commission. The project was started in 1985 and had a duration of five years. The results are recorded in the literature of Schlechtendahl

(1988 and 1989). The emphasis of this project was placed on the data exchange between various CAD systems, the exchange of CAD and FEM models and the conception of a neutral databank for CAD data.

For the realization of the data exchange reference architecture models for 3-D wireframe, surface, CSG and B-rep representations were established, and an interface for FEM applications was specified. For the description of CAD data structures, the specification language HDSL (High-level Data Specification Language) was designed. This language allows the expression of all essential elements of a CAD data structure with the help of special language constructs.

Within the framework of this project, an attempt was to be made to improve the deficiencies of IGES, SET and VDAFS which have the following problems:

- many of the specifications are imprecise,
- the file formats are not suited for an efficient processor implementation,
- the structuring capabilities of the file format for CAD data are not adequate,
- the scope of the interfaces is very restricted.

Work on this project was done in the following areas:

- investigation of CAD data structures and the presentation of 2-D and 3-D models by using new modeling techniques;
- specifying neutral file formats and providing access mechanisms;
- development of pre- and post-processors for interfacing various CAD systems;
- development of interface test methods, and aids for performing and documenting tests.

One special goal of the CAD*I project was the interaction with the STEP activities to develop solutions which could be directly implemented in STEP.

The publications of the CAD*I project are very complete and are well suited to give the reader not familiar with CAD/CAM interfaces a good insight into the general problems of software standards.

7.16.7 STEP (Standard for External Representation of Product Data)

The STEP (Standard for External Representation of Product Data) project is supported by the ISO-workgroup TC 184/SC4. It is also sometimes called Standard for Exchange of Product Definition Data. It is an effort by this group to develop an international standard for representing product model and a data exchange format for all information needed for the life cycle of the product. In general, it is a series of standards intended to provide a common mechanism for representing product model data throughout the life cycle of a product independent of any application software that may be used to process it. The basic principles of STEP are shown in Figure 7.37. The figure shows a computer-integrated manufacturing system where CAD data is tightly coupled via the STEP interface to the main production activities such as CAP, PP&C, CAM and CAQ. The following are the predecessors of STEP (which have been discussed earlier in this chapter): IGES (USA), SET (France) and VDAFS (Germany). See Figure 7.26 for these standards. It is estimated that by 1994, STEP will be replacing the standards mentioned above. The series of standards in STEP provide a neutral representation of product model data in the form of a set of integrated resources that support a complete and unambiguous definition of a product. The resource constructs are documented in a single product data language definition as discussed in Mason (1991).

There are five components in the STEP standard.

(1) The first component discusses the purpose of the standard. It contains:

(a) an overview describing the fundamentals of the entire standard,

(b) EXPRESS which is a formal language for specifying the information structures which are needed for defining the product model of STEP,

(c) framework for product modeling in which the basic methods of modeling are described.

(2) The second component defines standards for the implementation of STEP. It contains:

(a) physical-file implementations which define the rules according to which the data structures defined in EXPRESS are mapped on a segmental data file.

(3) The third component specifies the framework for conformance testing. This segment defines the rules according to which an implementation is tested for standard conformity.

(4) This component includes standards for product modeling (Figure 7.37). The models include:

(a) a presentation model which defines the type of presentation of information. For instance how the product can be visualized on a CRT or paper. Features like color, illumination, text font, view angles and so on are determined here, see Figure 7.37(a).

(b) Materials model is where the characteristics of material can be described. For instance an elasticity matrix, material-specific coefficients such as a coefficient of expansion, thermal conductivity, heat transfer are specified, see Figure 7.37(b).

(c) Tolerance model describes dimensional tolerances, geometric tolerances, form tolerances and so on, as shown in Figure 7.37(c).

(d) Surface model describes the specification for surface finish required, as in Figure 7.37(d).

(e) Form-feature model describes the object from the point of view of manufacturability whereby the form pattern is described or a method to produce the form element. The manufacturing relevant features can include holes, threads, keywords and so on, as shown in Figure 7.37(e).

(f) Shape representation model is based on the elements of geometry and

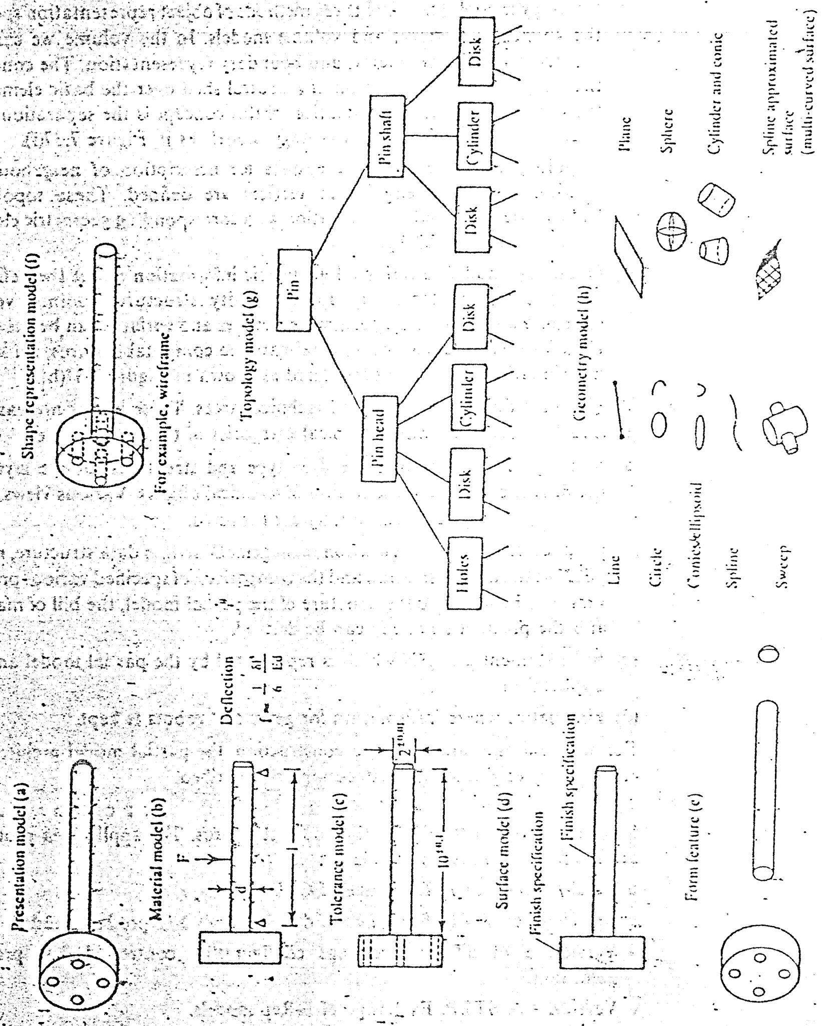

Figure 7.37
Models of STEP.

topology model. There are three methods of object representation specified: the wireframe, surface, and volume models. In the volume we can have constructive solid geometry, and boundary representation. The concept in these models is the formation of a neutral shell over the basic elements of the topology model. The intention of this concept is the separation of the operation models from the topology models as in Figure 7.37(f).

(g) Topology model defines the entities for description of neighbourhood relations. Surfaces, edges and vertices are defined. These topological features are described by association with corresponding geometric elements as seen in Figure 7.37(g).

(h) Geometry model contains all geometric information about the definition of lines, and surfaces. By specific entity structures, points, vectors, coordinate systems, transformations, curves and surfaces can be described. Free-form surfaces can be approximated to computable forms. All known mathematical forms are considered as shown in Figure 7.37(h).

(5) Standard for the specific standard technical areas. These norms are based on previously described models. Typical categories of these norms are:

(a) drafting, which describes the data type and structures for the layout of the drawing and the presentation of technical objects. Various views, cross sections, as well as dimensioning are captured.

(b) product structure configuration management defines data structure, makes possible the administration and the comparison of specified various product versions. From this data structure of the partial model, the bill of material and the product structure can be derived.

(c) finite element analysis which is represented by the partial model analysis application.

(d) kinematics where information for gears and robots is kept.

For ship building and building construction, the partial model architecture, engineering, engineering construction are described.

(6) Application protocols, which are used to give the user a certain amount of freedom, are still left open in the above standards. The application protocols are the final references to abide by:

- Version 1 of STEP: Exchange of CAD drawings.
- Version 2 of STEP: Exchange of drawing from 3-D product model.
- Version 3 of STEP: Exchange configuration controlled 3-D product definition.
- Version 4 of STEP: Exchange of B-Rep models.

In principle, the STEP development is done in three phases. In the first phase, an application-oriented information structure is provided. This structure is also called the application model, and is referred to as the application layer.

In the second phase, an information structure is provided by which all application models can be described with the help of the language EXPRESS. This phase is called the logical layer, and its output includes the information, objects, entities, attributes, associations and conditions.

In the third phase, the physical layer, the data formats and data files are defined in the Bachus-Naur-Form of the Wirth-Syntax Notation (WSN). The latter is a common language known in computer science for describing compilers.

7.16.8 EXPRESS language

EXPRESS is a definition for a formal language which was developed for use in STEP. It has syntactical similarity to Pascal and contains object-oriented features. The language can also be used in a broader sense for the specification of other information structures. The central role in EXPRESS is the entity. An entity has attributes which either constitute an entity or can derive from it. An entity can also have restrictions which define the range of values for the attributes.

With EXPRESS, a specification done for STEP can be tested formally. Furthermore, EXPRESS gives the fundamentals for software tools for the definition of product data and the description of data files.

An example of how EXPRESS is used in STEP is shown in [illegible] (1991) and is as follows:

*Right Circular Cylinder
A right circular cylinder is defined by an axis point at the center of [illegible] cylinder face, an axis, a height, and a radius. The faces are perpendicular [illegible] axis and are circular disks with the specified radius. The height [illegible] from the first circular face center in the positive direction [illegible] circular face center.

```
*)
  ENTITY right_circular_cylinder
SUBTYPE OF (primitive_with_one_axis)
radius    :   real;
position  :   axis 1_placement;
height    :   real;
  WHERE
WR1   :   radius > 0;
WR2   :   height > 0;
  END_ENTITY;
  (*
ATTRIBUTE DEFINITIONS:
Radius: The radius of the cylinder
Position: The location of a point on the axis and the direction of the axis
Height: The distance between the bases of the cylinder

PROPOSITIONS:
(a) The radius must be positive.
(b) The height must be positive.
```